

死磕设计模式之如何抽象策略模式

策略模式是什么

策略模式在 GoF 的《设计模式》一书中，是这样定义的：

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

策略设计模式（Strategy Pattern）是一种面向对象设计模式，它定义了一系列算法，并将每个算法封装起来，使它们可以相互替换。这种模式使得算法可以独立于使用它们的客户端而变化。

策略设计模式包含三个主要的角色：

1. 环境（Context）：持有一个策略对象，并调用其算法。
2. 策略（Strategy）：定义了一组算法，并将每个算法封装起来，使它们可以相互替换。
3. 具体策略（ConcreteStrategy）：实现了策略接口，提供了具体的算法实现。

在策略设计模式中，环境持有一个策略对象，并通过调用策略的算法来完成具体的任务。策略对象可以根据需要进行替换，从而实现不同的算法实现，并且可以在运行时动态地更改策略对象。

看到上面的介绍可能不太明白策略模式具体为何物，这里会从最基本的代码说起，一步一步彻底掌握此模式。

优惠类型实战

下述代码可能大家都能联想出对应的业务，**根据对应的优惠类型，对价格作出相应的优惠。**

```

public class Example {

    public static void main(String[] args) {
        Double discount = discount("1", 20D);
        System.out.println(discount);
    }

    public static Double discount(String type, Double price) {
        if (Objects.equals(type, "1")) { // 优惠 9.5 折
            return price * 0.95;
        } else if (Objects.equals(type, "2")) { // 优惠 9 折
            return price * 0.9;
        } else if (Objects.equals(type, "3")) { // 优惠 8.5 折
            return price * 0.85;
        } else {
            // 无优惠
            return price;
        }
    }
}

```

这段代码是能够满足项目中业务需求的，而且很多已上线生产环境的代码也有这类代码。但是，这一段代码存在存在两个弊端：

1. 代码的复杂性，正常业务代码逻辑肯定会比这个代码块复杂很多，这也就 **导致了 if-else 的分支以及代码数量过多**。这种方式可以通过将代码拆分成独立函数或者拆分成类来解决。
2. 开闭原则，价格优惠肯定会 **随着不同的时期作出不同的改变**，或许新增、删除或修改。如果在一个函数中修改无疑是件恐怖的事情，想想可能多个开发者分别进行开发，杂乱无章的注释，混乱的代码逻辑等情况十有八九会发生。

如何运用策略模式优化上述代码，使程序设计看着简约、可扩展等特性。

1. 简化代码的复杂性，将不同的优惠类型定义为不同的策略算法实现类。
2. 保证开闭原则，增加程序的健壮性以及可扩展性。

将上述代码块改造为策略设计模式，大致需要三个步骤。

1. 定义抽象策略接口，因为业务使用接口而不是具体的实现类的话，便可以灵活的替换不同的策略；
2. 定义具体策略实现类，实现自抽象策略接口，其内部封装具体的业务实现；
3. 定义策略工厂，封装创建策略实现（算法），对客户端屏蔽具体的创建细节。

```

public interface DiscountStrategy { // 定义策略抽象接口
    Double discount(Double price);
}

public class Discount95Strategy implements DiscountStrategy { // 95折优惠策略类
    @Override
    public Double discount(Double price) { return price * 0.95; }
}

public class Discount9Strategy implements DiscountStrategy { // 9折优惠策略类
    @Override
    public Double discount(Double price) { return price * 0.9; }
}

public class DiscountStrategyFactory {
    private static final Map<String, DiscountStrategy> discountStrategies = new HashMap(); // 策略存储容器

    static {
        discountStrategies.put("1", new Discount95Strategy());
        discountStrategies.put("2", new Discount9Strategy());
    }

    public static DiscountStrategy chooseStrategy(String type) {
        return discountStrategies.get(type);
    }
}

```

目前把抽象策略接口、具体的策略实现类以及策略工厂都已经创建了，现在可以看一下客户端需要如何调用，又是如何对客户端屏蔽具体实现细节的。

```

public class DiscountApplication {

    public static void main(String[] args) {
        // 从抽象策略工厂中选择具体的策略
        DiscountStrategy discountStrategy = DiscountStrategyFactory.chooseStrategy("1");

        // 执行具体策略
        Double discount = discountStrategy.discount(100);

        // 输出策略执行结果
        System.out.println("优惠后金额 :: " + discount);

        /**
         * 输出结果如下:
         *
         * 优惠后金额 :: 9.5
         */
    }
}

```

根据代码块图片得知，具体策略类是从策略工厂中获取，确实是取消了 if-else 设计，**在工厂中使用 Map 存储策略实现**。获取到策略类后执行具体的优惠策略方法就可以获取优惠后的金额。

通过分析大家得知，目前这种设计确实将应用代码的复杂性降低了。**如果新增一个优惠策略，只需要新增一个策略算法实现类即可。**但是，添加一个策略算法实现，**意味着需要改动策略工厂中的代码**，还是不符合开闭原则。

如何完整实现符合开闭原则的策略模式，需要借助 Spring 的帮助，详细案例请继续往下看。

策略模式结合 Spring

最近项目中设计的一个功能用到了策略模式，分为两类角色，笔者负责定义抽象策略接口以及策略工厂，不同的策略算法需要各个业务方去实现，可以联想到上文中的优惠券功能。因为是 Spring 项目，所以都是按照 Spring 的方式进行处理。

```
public interface DiscountStrategy {
    Double discount(Double price); // 定义抽象策略接口
    String mark(); // 定义策略算法标示
}

@Component
public class Discount95Strategy implements DiscountStrategy {
    @Override
    public Double discount(Double price) { return price * 0.95; }

    @Override
    public String mark() { return "1"; }
}

@Component
public class Discount9Strategy implements DiscountStrategy {
    @Override
    public Double discount(Double price) { return price * 0.9; }

    @Override
    public String mark() { return "2"; }
}
```

可以看到，比对上面的示例代码，有两处明显的变化：

1. 抽象策略接口中，新定义了 mark() 接口，此接口用来标示算法的唯一性；
2. 具体策略实现类，使用 @Component 修饰，将对象本身交由 Spring 进行管理。

小贴士：为了阅读方便，mark() 返回直接使用字符串替代，读者朋友在返回标示时最好使用枚举定义。

接下来继续查看抽象策略工厂如何改造，才能满足开闭原则。

```
@Component
public class DiscountStrategyFactory implements InitializingBean {

    @Autowired
    private ApplicationContext applicationContext;

    private final Map<String, DiscountStrategy> discountStrategies = new HashMap();

    public DiscountStrategy chooseStrategy(String type) {
        return discountStrategies.get(type);
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        // 从 IOC 容器中获取 DiscountStrategy 类型的 Bean 对象，而这些 Bean 对象就是具体的策略算法实现类
        Map<String, DiscountStrategy> discountStrategyMap = applicationContext.getBeansOfType(DiscountStrategy.class);

        // 将具体的策略算法标示以及本身添加到我们自定义的 Map 中，方便屏蔽客户端感知调用
        discountStrategyMap.forEach((key, val) -> discountStrategies.put(val.mark(), val));
    }
}
```

通过 InitializingBean 接口实现中调用 IOC 容器查找对应策略实现，随后将策略实现 mark() 方法返回值作为 key，策略实现本身作为 value 添加到 Map 容器中等待客户端的调用。

```
@SpringBootTest
class StrategyApplicationTests {

    @Autowired
    private DiscountStrategyFactory discountStrategyFactory;

    @Test
    void contextLoads() {
        // 通过优惠策略工厂获取类型为 1 的具体策略算法
        DiscountStrategy discountStrategy = discountStrategyFactory.chooseStrategy("1");

        // 根据策略算法获取优惠后的价格
        Double discount = discountStrategy.discount(100);

        System.out.println(discount);
    }
}
```

这里使用的 SpringBoot 测试类，注入策略工厂 Bean，通过策略工厂选择出具体的策略算法类，继而通过算法获取到优惠后的价格。

总结下本小节，我们通过和 Spring 结合的方式，通过策略设计模式对文初的代码块进行了两块优化：应对代码的复杂性，让其满足开闭原则。

更具体一些呢就是 **通过抽象策略算法类减少代码的复杂性，继而通过 Spring 的一些特性同时满足了开闭原则**，现在来了新需求只要添加新的策略类即可，健壮易扩展。

策略模式优缺点

策略设计模式的优点包括：

1. 提高了代码的灵活性和可维护性：由于算法的实现与使用相分离，使得代码的灵活性和可维护性得到提高。当需要修改或添加新的算法时，只需要定义新的策略类，并将其传递给环境类即可，而无需修改环境类的代码。
2. 提高了代码的复用性：策略设计模式将算法的实现封装在策略类中，使得算法可以被多个客户端重复使用，从而提高了代码的复用性。
3. 可以动态地切换算法：策略设计模式将算法封装在策略类中，使得可以在运行时动态地更改算法，从而实现不同的功能和行为。这样可以使得程序更加灵活，适应不同的需求和变化。
4. 算法实现与使用相分离：策略设计模式将算法的实现与使用相分离，使得代码更加清晰、简洁、易于维护和扩展。由于算法的实现被封装在策略类中，客户端只需要关注如何选择和使用不同的算法即可，这样可以使得代码更加易于理解和维护。
5. 可以避免使用大量的条件语句：在某些情况下，需要根据不同的条件来选择不同的算法，这可能会导致代码中出现大量的条件语句，使得代码难以维护和扩展。而策略设计模式可以避免这种情况的发生，使得代码更加简洁和易于维护。

需要注意的是，在使用策略设计模式时需要注意以下几点：

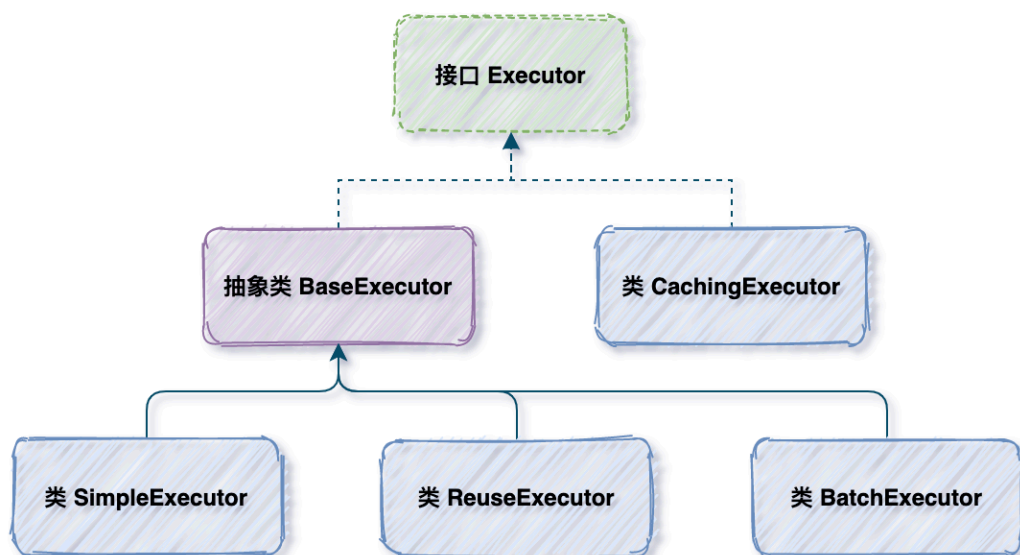
1. 策略类之间应该是相互独立的，彼此之间不应该有任何依赖关系。这样才能确保算法的选择和替换可以在运行时动态地进行，同时也可以避免代码的耦合度过高。
2. 策略接口应该尽可能地简单和通用，以便于不同的策略实现类可以共用同一个接口。这样可以提高代码的复用性和灵活性，同时也可以避免接口过于复杂和难以维护。
3. 策略设计模式适用于需要在运行时动态切换算法的场景，如果算法的实现不需要动态切换，或者算法的实现较为简单，策略设计模式可能会显得过于复杂。因此，在选择使用策略设计模式时，需要根据具体的需求和场景进行判断和选择。
4. 策略设计模式可以与其他设计模式结合使用，例如工厂方法模式、单例模式等。这样可以进一步提高代码的灵活性和可维护性，同时也可以避免代码的复杂度过高。

框架源码应用

自己用肯定觉得不够，必要时还得看看设计开源框架源码的大佬们如何在代码中运用策略模式的。

在马哥了解中，JDK、Spring、SpringMvc、Mybatis、Dubbo 等等都运用了策略设计模式，这里就以 Mybatis 举例说明。

Mybatis 中 Executor 代表执行器，负责增删改查的具体操作。其中用到了两种设计模式，模版方法以及策略模式。



- Executor 代表了抽象策略接口，刚才说到的模版方法模式源自 BaseExecutor。
- Configuration 代表策略工厂，负责创建具体的策略算法实现类。
- SimpleExecuto、ReuseExecutor... 表示封装了具体的策略算法实现类。

```

public Executor newExecutor(Transaction transaction, ExecutorType executorType) {
    executorType = executorType == null ? defaultExecutorType : executorType;
    executorType = executorType == null ? ExecutorType.SIMPLE : executorType;

    Executor executor;
    if (ExecutorType.BATCH == executorType) {
        executor = new BatchExecutor(this, transaction);
    } else if (ExecutorType.REUSE == executorType) {
        executor = new ReuseExecutor(this, transaction);
    } else {
        executor = new SimpleExecutor(this, transaction);
    }

    if (cacheEnabled) {
        executor = new CachingExecutor(executor);
    }

    executor = (Executor) interceptorChain.pluginAll(executor);
    return executor;
}

```

上述代码块发生在 Configuration 类中创建执行器 Executor，通过 executorType 判断创建不同的策略算法。

上述代码块并没有彻底消除 if-else，因为 Mybatis 中执行器策略基本是固定的，也就是说它只会有这些 if-else 判断，基本不会新增或修改。如果非要消除 if-else，可以这么搞，这里写一下伪代码。

```

static final Map<ExecutorType, Executor> executors = new HashMap();

static {
    executors.put(ExecutorType.SIMPLE, new SimpleExecutor(this, transaction));
    executors.put(ExecutorType.BATCH, new BatchExecutor(this, transaction));
    ...
}

public Executor newExecutor(Transaction transaction, ExecutorType executorType) {
    executorType = executorType == null ? defaultExecutorType : executorType;
    executorType = executorType == null ? ExecutorType.SIMPLE : executorType;

    return executors.get(executorType);
}

```

这种方式叫做“查表法”，通过策略工厂实现消除 if-else 分支。最后，Mybatis 太过详细的设计这里不再赘述，有兴趣的小伙伴可以去把源码下载啃一啃。

到了这里可能有读者看出了问题，策略模式就算消除了 if-else 但是如果添加新的策略类，不还是会违反开闭原则么？

没错，因为 Mybatis 本身没有引入 Spring 依赖，**所以没有办法借助 IOC 容器实现开闭原则**。Spring 是一种开闭原则解决方式，那还有没有别的解决方式？

解决方式有很多，开闭原则核心就是 **对原有代码修改关闭，对新增代码开放**。可以通过扫描指定包下的自定义注解亦或者通过 instanceof 判断是否继承自某接口都可以。

策略模式抽象

可能细心的小伙伴会发现一个问题，当业务使用越来越多的情况下，重复定义 `DiscountStrategy` 以及 `DiscountStrategyFactory` 会增加系统冗余代码量。

可以考虑将这两个基础类抽象出来，作为基础组件库中的通用组件，供所有系统下的业务使用，从而避免代码冗余。

定义抽象策略处理接口，添加有返回值和无返回值接口。

```
package org.opengoofy.congomall.springboot.starter.designpattern.strategy;

/**
 * 策略执行抽象
 */
public interface AbstractExecuteStrategy<REQUEST, RESPONSE> {

    /**
     * 执行策略标识
     */
    String mark();

    /**
     * 执行策略
     *
     * @param requestParam 执行策略入参
     */
    default void execute(REQUEST requestParam) {

    }

    /**
     * 执行策略，带返回值
     */
}
```

```

    *
    * @param requestParam 执行策略入参
    * @return 执行策略后返回值
    */
    default RESPONSE executeResp(REQUEST requestParam) {
        return null;
    }
}

```

添加策略选择器，通过订阅 Spring 初始化事件执行扫描所有策略模式接口执行器，并根据 mark 方法定义标识添加到 `abstractExecuteStrategyMap` 容器中。

客户端在实际业务中使用 `AbstractStrategyChoose#choose` 即可完成策略模式实现。

```

package org.opengoofy.congomall.springboot.starter.designpattern.strategy;

import org.opengoofy.congomall.springboot.starter.base.ApplicationContextHolder;
import org.opengoofy.congomall.springboot.starter.base.init.ApplicationInitializingEvent;
import org.opengoofy.congomall.springboot.starter.convention.exception.ServiceException;
import org.springframework.context.ApplicationListener;

import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

/**
 * 策略选择器
 */
public class AbstractStrategyChoose implements
    ApplicationListener<ApplicationInitializingEvent> {

    /**
     * 执行策略集合
     */
    private final Map<String, AbstractExecuteStrategy> abstractExecuteStrategyMap = new
    HashMap<>();

    /**
     * 根据 mark 查询具体策略
     */
}

```

```

    * @param mark 策略标识
    * @return 实际执行策略
    */
    public AbstractExecuteStrategy choose(String mark) {
        return Optional.ofNullable(abstractExecuteStrategyMap.get(mark)).orElseThrow(() -
> new ServiceException(String.format("[%s] 策略未定义", mark)));
    }

    /**
     * 根据 mark 查询具体策略并执行
     *
     * @param mark 策略标识
     * @param requestParam 执行策略入参
     * @param <REQUEST> 执行策略入参范型
     */
    public <REQUEST> void chooseAndExecute(String mark, REQUEST requestParam) {
        AbstractExecuteStrategy executeStrategy = choose(mark);
        executeStrategy.execute(requestParam);
    }

    /**
     * 根据 mark 查询具体策略并执行，带返回结果
     *
     * @param mark 策略标识
     * @param requestParam 执行策略入参
     * @param <REQUEST> 执行策略入参范型
     * @param <RESPONSE> 执行策略出参范型
     * @return
     */
    public <REQUEST, RESPONSE> RESPONSE chooseAndExecuteResp(String mark, REQUEST
requestParam) {
        AbstractExecuteStrategy executeStrategy = choose(mark);
        return (RESPONSE) executeStrategy.executeResp(requestParam);
    }

    @Override
    public void onApplicationEvent(ApplicationInitializingEvent event) {
        Map<String, AbstractExecuteStrategy> actual =
ApplicationContextHolder.getBeansOfType(AbstractExecuteStrategy.class);
        actual.forEach((beanName, bean) -> {
            AbstractExecuteStrategy beanExist =

```

```
abstractExecuteStrategyMap.get(bean.mark());
    if (beanExist != null) {
        throw new ServiceException(String.format("[%s] Duplicate execution
policy", bean.mark()));
    }
    abstractExecuteStrategyMap.put(bean.mark(), bean);
});
}
}
```

这两个实现已经被放置在基础组件库中，如果业务需要使用策略模式，则无需重新定义。

文末总结

总的来说，策略模式是一种非常有用的设计模式，它可以帮助我们在运行时动态地选择不同的算法实现，从而提高代码的灵活性、可维护性和可扩展性。

通过将算法实现封装在不同的策略类中，并定义一个策略接口来描述算法，策略模式使得客户端可以独立于算法实现进行编程，同时也可以方便地更换算法实现或者添加新的算法实现。

策略模式的优点包括提高代码的灵活性、可维护性、可扩展性和复用性，同时也可以避免使用大量的条件语句和提高代码的可读性。

需要注意的是，在使用策略模式时需要遵循一些设计原则，例如开闭原则、单一职责原则和依赖倒置原则等，以确保代码的高内聚性和低耦合度。

此外，策略模式适用于需要在运行时动态选择不同算法实现的场景，如果算法实现不需要动态切换或者算法实现较为简单，策略模式可能会显得过于复杂。

因此，在使用策略模式时需要根据具体的需求和场景进行判断和选择。

更新: 2023-07-17 15:18:16

原文: <<https://www.yuque.com/magestack/12306/alawl3uq9rqegate>>